Week 5 – Payment Transaction Management System

Database: OrderManagementDB | **Prepared by:** Shree Pranava Ganesh | **Date:** 28 October 2026

Objective

This task adds a **Payment** module to the existing Order Management System. Each order can have at most one payment record. The module stores the payment mode and status, supports transaction retries, and produces payment and revenue analysis reports linked to the existing `Customer` and `Orders` tables.

---

1. Payment Table Design

1.1 Table: Payment

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Payment_Status	Transaction_Amount
1	1001	2026-10-01	Credit Card	Success	59000.00
2	1002	2026-10-03	UPI	Success	27300.00
3	1003	2026-10-05	Debit Card	Success	3800.00
4	1004	2026-10-08	Net Banking	Failed	39500.00
5	1005	2026-10-10	Cash on Delivery	Success	2300.00
6	1006	2026-10-12	UPI	Success	5500.00
7	1007	2026-10-17	UPI	Success	3000.00
8	1008	2026-10-18	UPI	Success	2050.00
9	1009	2026-10-20	Cash on Delivery	Pending	8200.00
10	1010	2026-10-22	Net Banking	Success	1500.00
NULL	NULL	NULL	NULL	NULL	NULL

1.2 Design decisions explained

- **At most one payment per order:** `Order_ID` is marked `UNIQUE`. The database itself rejects a second payment row for the same order, so no application code is needed to enforce this rule.
- **Foreign key to Orders:** a payment cannot exist for an order that does not exist, which keeps the payment records linked to the existing system.
- **CHECK on Payment_Mode and Payment_Status:** only the five allowed modes and three allowed statuses can be stored, which prevents spelling variations such as "upi" or "Paid".
- **DEFAULT 'Pending':** a new transaction starts as Pending until it is confirmed or fails.
- **CHECK on Transaction_Amount:** zero or negative payments are rejected.
- **Retries update the same row:** because of the UNIQUE rule, a retry after a failure updates the existing payment (new date, mode and status) instead of adding a second row.

1.3 Creation script

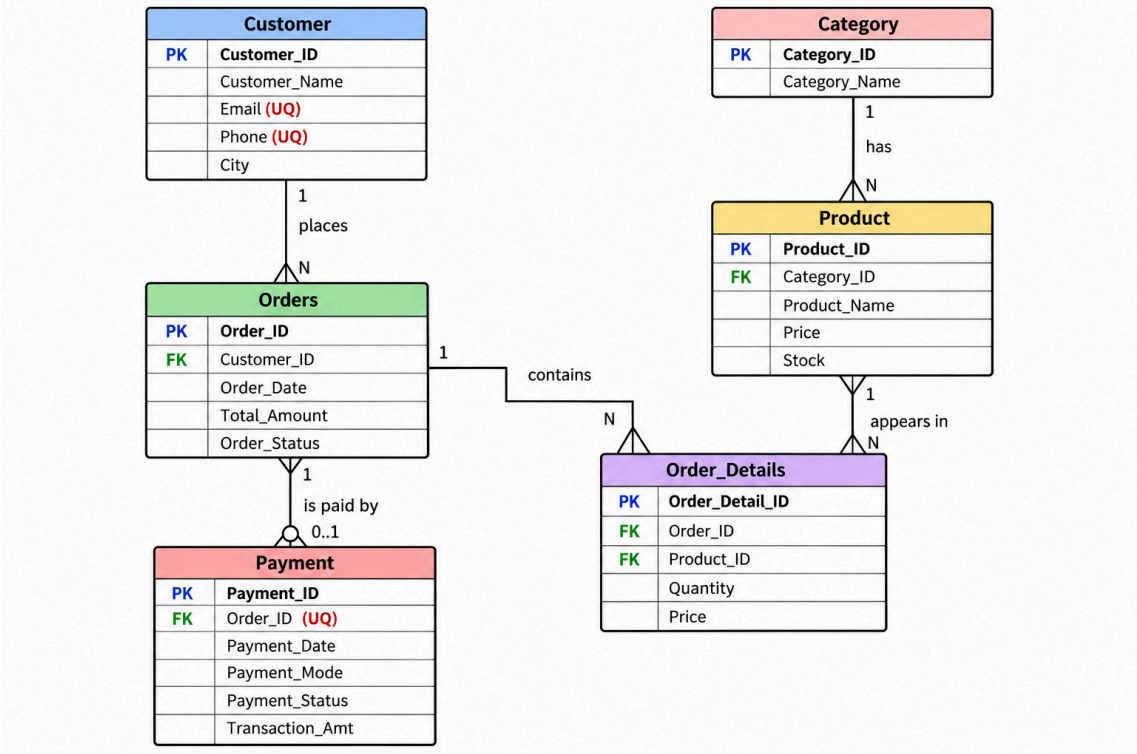
```
CREATE TABLE Payment (
  Payment_ID       INT PRIMARY KEY,
  Order_ID         INT NOT NULL UNIQUE,
```

```
  Payment_Date       DATE NOT NULL,
  Payment_Mode       VARCHAR(30) NOT NULL,
  Payment_Status     VARCHAR(20) NOT NULL DEFAULT 'Pending',
  Transaction_Amount DECIMAL(10,2) NOT NULL,
  CONSTRAINT fk_payment_order
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
  CONSTRAINT chk_payment_mode
    CHECK (Payment_Mode IN ('UPI','Credit Card','Debit Card',
                            'Net Banking','Cash on Delivery')),
  CONSTRAINT chk_payment_status
    CHECK (Payment_Status IN ('Pending','Success','Failed')),
  CONSTRAINT chk_transaction_amount
    CHECK (Transaction_Amount > 0)
);
```

---

2. Updated ER Diagram (Order–Payment Relationship)

The Payment table connects to Orders with a **one-to-one** relationship (one order has at most one payment). The rest of the Order Management System is unchanged.



Relationships

Relationship	Cardinality	Foreign key	Meaning
Customer → Orders	1 : N	Orders.Customer_ID	One customer places many orders

Relationship	Cardinality	Foreign key	Meaning
Orders → Order_Details	1 : N	Order_Details.Order_ID	One order has many line items
Orders → Payment	1 : 0..1	Payment.Order_ID (UNIQUE)	One order has at most one payment
Category → Product	1 : N	Product.Category_ID	One category groups many products

---

3. Sample Customer, Order & Payment Data

3.1 Customer

Customer_ID	Customer_Name	Email	Phone	City
1	Arun Kumar	arun@gmail.com	9876543210	Chennai
2	Priya Sharma	priya@gmail.com	9876543211	Coimbatore
3	Rahul Raj	rahul@gmail.com	9876543212	Madurai
4	Divya S	divya@gmail.com	9876543213	Trichy
5	Karthik R	karthik@gmail.com	9876543214	Thoothukudi
NULL	NULL	NULL	NULL	NULL

3.2 Orders

Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
1001	1	2026-10-01	59000.00	Delivered
1002	2	2026-10-03	27300.00	Shipped
1003	1	2026-10-05	3800.00	Delivered
1004	3	2026-10-08	39500.00	Pending
1005	4	2026-10-10	2300.00	Delivered
1006	5	2026-10-12	5500.00	Shipped
1007	2	2026-10-15	3000.00	Pending
1008	1	2026-10-18	2050.00	Delivered
1009	3	2026-10-20	8200.00	Shipped
1010	4	2026-10-22	1500.00	Delivered
1011	5	2026-10-23	800.00	Cancelled
NULL	NULL	NULL	NULL	NULL

3.3 Payment – initial data (9 rows)

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Payment_Status	Transaction_Amount
1	1001	2026-10-01	Credit Card	Success	59000.00
2	1002	2026-10-03	UPI	Success	27300.00
3	1003	2026-10-05	Debit Card	Success	3800.00
4	1004	2026-10-08	Net Banking	Failed	39500.00
5	1005	2026-10-10	Cash on Delivery	Success	2300.00
6	1006	2026-10-12	UPI	Success	5500.00
7	1007	2026-10-17	UPI	Success	3000.00
8	1008	2026-10-18	UPI	Success	2050.00
9	1009	2026-10-20	Cash on Delivery	Pending	8200.00
10	1010	2026-10-22	Net Banking	Success	1500.00
NULL	NULL	NULL	NULL	NULL	NULL

3.5 Transaction operations performed

Operation	SQL	Why
Insert new payment	INSERT for order 1010, status Pending	A new transaction always starts as Pending
Retrieve one order's payment	SELECT ... WHERE Order_ID = 1005	Shows the payment details of a specific order
Confirm payment	UPDATE status to Success WHERE Order_ID = 1010 AND status = 'Pending'	Pending → Success
Retry a failed payment	UPDATE mode = 'UPI', date, status = 'Success' WHERE Order_ID = 1007 AND status = 'Failed'	Retry updates the same row because Order_ID is UNIQUE
Integrity check	JOIN Payment with Orders where amounts differ	Returns 0 rows: every payment equals its order total

3.6 Constraint tests

Test	Statement	Expected error
Second payment for one order	INSERT with Order_ID 1001 again	UNIQUE constraint violation (duplicate entry)
Invalid payment mode	INSERT with mode 'Bitcoin'	CHECK constraint chk_payment_mode violated
Payment for a missing order	INSERT with Order_ID 9999	FOREIGN KEY constraint fails

---

4. Payment Transaction & Revenue Analysis Reports

All figures are calculated from the final data in Section 3.4.

4.1 Successful payments

```
SELECT Payment_ID, Order_ID, Payment_Date, Payment_Mode, Transaction_Amount
FROM Payment WHERE Payment_Status = 'Success' ORDER BY Payment_Date;
```

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Transaction_Amount
1	1001	2026-10-01	Credit Card	59000.00
2	1002	2026-10-03	UPI	27300.00
3	1003	2026-10-05	Debit Card	3800.00
5	1005	2026-10-10	Cash on Delivery	2300.00
6	1006	2026-10-12	UPI	5500.00
7	1007	2026-10-17	UPI	3000.00
8	1008	2026-10-18	UPI	2050.00
10	1010	2026-10-22	Net Banking	1500.00
NULL	NULL	NULL	NULL	NULL

4.2 Transactions by status

```
SELECT Payment_Status, COUNT(*) AS Number_Of_Transactions,
      SUM(Transaction_Amount) AS Total_Amount
FROM Payment GROUP BY Payment_Status ORDER BY Number_Of_Transactions DESC;
```

Payment_Status	Number_Of_Transactions	Total_Amount
Success	8	104450.00
Failed	1	39500.00
Pending	1	8200.00

4.3 Transactions and revenue by payment mode

```
SELECT Payment_Mode, COUNT(*) AS Total_Transactions,
  SUM(CASE WHEN Payment_Status = 'Success' THEN 1 ELSE 0 END) AS Successful_Transactions,
  SUM(CASE WHEN Payment_Status = 'Success' THEN Transaction_Amount ELSE 0 END) AS
Successful_Revenue
FROM Payment GROUP BY Payment_Mode ORDER BY Successful_Revenue DESC;
```

Payment_Mode	Total_Transactions	Successful_Transactions	Successful_Revenue
Credit Card	1	1	59000.00
UPI	4	4	37850.00
Debit Card	1	1	3800.00
Cash on Delivery	2	1	2300.00
Net Banking	2	1	1500.00

4.4 Most preferred payment method

UPI, with 4 transactions (orders 1002, 1006, 1007 and 1008). The `HAVING COUNT(*) = (SELECT MAX(...))` form is used so that tied modes would all be shown, which `LIMIT 1` would not do.

4.5 Key revenue and average figures

Metric	Query	Result
Total successful revenue	SUM(Transaction_Amount) WHERE status = 'Success'	₹1,04,450.00
Average transaction (all)	ROUND(AVG(Transaction_Amount), 2)	₹15,215.00
Average transaction (successful)	AVG over Success rows only	₹13,056.25

4.6 Customer payment history

Customer	Order_ID	Payment_Date	Mode	Status	Amount
Arun Kumar	1001	2026-10-01	Credit Card	Success	59000.00
Arun Kumar	1003	2026-10-05	Debit Card	Success	3800.00
Arun Kumar	1008	2026-10-18	UPI	Success	2050.00
Priya Sharma	1002	2026-10-03	UPI	Success	27300.00
Priya Sharma	1007	2026-10-17	UPI	Success	3000.00
Rahul Raj	1004	2026-10-08	Net Banking	Failed	39500.00
Rahul Raj	1009	2026-10-20	Cash on Delivery	Pending	8200.00
Divya S	1005	2026-10-10	Cash on Delivery	Success	2300.00
Divya S	1010	2026-10-22	Net Banking	Success	1500.00
Karthik R	1006	2026-10-12	UPI	Success	5500.00

4.7 Orders with failed or pending payments

Order_ID	Customer	Order_Status	Order Total	Payment_Mode	Payment_Status
1004	Rahul Raj	Pending	39500.00	Net Banking	Failed
1009	Rahul Raj	Shipped	8200.00	Cash on Delivery	Pending

4.8 Key findings

1. ₹1,04,450 of the ₹1,52,150 total (about 68.6%) has been collected successfully.
2. Both outstanding payments belong to Rahul Raj: a failed ₹39,500 Net Banking payment (order 1004) and a pending ₹8,200 Cash on Delivery payment (order 1009).
3. UPI is the most used method (4 of 10 transactions), while Credit Card brings the most revenue from a single transaction (the ₹59,000 Laptop order).
4. The retry of order 1007 through UPI turned a failed transaction into a success, which shows the failure-and-retry workflow.
5. Every order has a payment record, so the check for orders without any payment returns no rows.

Conclusion

The `Payment` table extends the Order Management System with a validated, one-payment-per-order design. Primary key, foreign key, UNIQUE, NOT NULL, DEFAULT and CHECK constraints keep the data consistent. The status updates and retry show the full life of a transaction, and the JOIN and GROUP BY reports give clear insight into payment methods, collected revenue and outstanding payments.